

# Kubernetes for Test Automation

## A Complete Phase-Wise Guidebook

From Zero Knowledge → Production-Ready Automation Pipelines

**Phase 1**  
**Foundation**  
Concepts & Setup

**Phase 2**  
**Core Objects**  
Pods, Deploy,  
Services

**Phase 3**  
**Test Workflows**  
Jobs & ConfigMaps

**Phase 4**  
**CI/CD**  
Pipelines & Helm

**Phase 5**  
**Production**  
Scale & Monitor

### About This Guide

This guidebook is written specifically for **test automation engineers** who have little or no prior Kubernetes knowledge. Every concept is explained in plain English first, then backed with **real YAML/CLI examples** that you can copy and run immediately. You will find **Hands-On Labs** at the end of every major section so you build muscle memory, not just theory. By the time you finish all five phases you will be able to spin up a full **Selenium/Playwright test grid on Kubernetes**, wire it into a CI/CD pipeline, and monitor it — all without depending on any other resource.

|                 |                                                                          |
|-----------------|--------------------------------------------------------------------------|
| Author Role     | Senior Technical Architect, Kubernetes                                   |
| Target Audience | QA / Test Automation Engineers (0 → Production)                          |
| Covers          | K8s v1.28+   Docker   Selenium Grid   Playwright   Helm   GitHub Actions |
| Sections        | 5 Phases • 20+ Hands-On Labs • 50+ Code Examples                         |

---

# Table of Contents

---

|                |                                                           |           |
|----------------|-----------------------------------------------------------|-----------|
| <b>Phase 1</b> | <b>Foundation — Understanding the Kubernetes Universe</b> | <b>3</b>  |
|                | 1.1 What is Kubernetes and Why Should Testers Care?       | 3         |
|                | 1.2 Core Concepts Dictionary                              | 4         |
|                | 1.3 Kubernetes Architecture (with Diagrams)               | 5         |
|                | 1.4 Installing Your Local Cluster (minikube / kind)       | 6         |
|                | 1.5 Your First kubectl Commands                           | 7         |
|                | ■ Lab 1: Hello Kubernetes!                                | 8         |
| <b>Phase 2</b> | <b>Core Kubernetes Objects</b>                            | <b>9</b>  |
|                | 2.1 Pods — The Atom of Kubernetes                         | 9         |
|                | 2.2 Deployments — Self-Healing Apps                       | 10        |
|                | 2.3 Services — Networking Made Simple                     | 11        |
|                | 2.4 Namespaces — Keeping Things Organised                 | 12        |
|                | 2.5 ConfigMaps & Secrets                                  | 13        |
|                | ■ Lab 2: Deploy a Test App                                | 14        |
| <b>Phase 3</b> | <b>Running Test Automation Workloads</b>                  | <b>15</b> |
|                | 3.1 Kubernetes Jobs & CronJobs for Tests                  | 15        |
|                | 3.2 Selenium Grid on Kubernetes                           | 16        |
|                | 3.3 Playwright in Kubernetes                              | 18        |
|                | 3.4 Persistent Volumes for Test Reports                   | 19        |
|                | ■ Lab 3: Run Selenium Tests in a Pod                      | 20        |
| <b>Phase 4</b> | <b>CI/CD Integration</b>                                  | <b>21</b> |
|                | 4.1 GitHub Actions + kubectl                              | 21        |
|                | 4.2 Helm — Package Manager for K8s                        | 22        |
|                | 4.3 Test Result Artefacts & Reporting                     | 23        |
|                | ■ Lab 4: Full CI Pipeline                                 | 24        |
| <b>Phase 5</b> | <b>Production Readiness</b>                               | <b>25</b> |
|                | 5.1 Resource Limits & Requests                            | 25        |
|                | 5.2 Health Checks (Liveness / Readiness Probes)           | 26        |
|                | 5.3 Monitoring with Prometheus & Grafana                  | 27        |
|                | 5.4 Debugging Cheat Sheet                                 | 28        |
|                | ■ Lab 5: Production-Grade Test Cluster                    | 29        |



# Phase 1

## Foundation Understanding the Kubernetes Universe

### 1.1 What is Kubernetes and Why Should Testers Care?

Imagine you have 50 test scripts and you need to run them all at the same time — but your laptop has only 8 GB of RAM. Kubernetes solves exactly this. It lets you spread your tests across **many machines (nodes)** and manages everything for you: starting containers, restarting failed ones, sharing network ports, and cleaning up when done.

#### ■ Key Concept

Kubernetes (also written as **K8s**) is an open-source platform that **orchestrates containers**. 'Orchestrate' simply means: schedule them, run them, watch over them, and replace them if they crash. Think of it as a very smart manager for your Docker containers.

### Why does a Test Engineer need Kubernetes?

| Problem Without K8s                        | How K8s Solves It                           |
|--------------------------------------------|---------------------------------------------|
| Running 100 tests serially takes hours     | Run them in parallel pods — cut time 10×    |
| Browser containers crash, tests fail       | K8s auto-restarts the container for you     |
| Need Selenium Grid on every machine        | Deploy once, use from anywhere in CI        |
| Test reports disappear when container dies | Persistent Volumes keep data safe           |
| Different environments = different configs | ConfigMaps & Secrets manage configs cleanly |

### 1.2 Core Concepts Dictionary

Before you touch any YAML file, let's make sure every term is crystal clear. These are the 10 words you will see everywhere in Kubernetes:

|                  |                                                                                                              |
|------------------|--------------------------------------------------------------------------------------------------------------|
| <b>Container</b> | A running instance of a Docker image. Think of it as a very lightweight virtual machine that runs one app.   |
| <b>Pod</b>       | The smallest unit K8s manages. Usually holds 1 container. Pods are temporary — they can die and be replaced. |

|                   |                                                                                                                       |
|-------------------|-----------------------------------------------------------------------------------------------------------------------|
| <b>Node</b>       | A physical or virtual machine in your cluster. Pods run on nodes.                                                     |
| <b>Cluster</b>    | A group of nodes managed by Kubernetes. Typically 1 master + several worker nodes.                                    |
| <b>Deployment</b> | A blueprint that says 'I want 3 copies of this pod always running'. K8s keeps that promise.                           |
| <b>Service</b>    | A stable network address for your pods. Even when pods restart (and their IP changes), the Service IP stays the same. |
| <b>Namespace</b>  | A virtual folder inside the cluster. Keeps dev, staging, and test resources separated.                                |
| <b>ConfigMap</b>  | Stores non-sensitive config like URLs, timeouts. Injected into pods as env vars or files.                             |
| <b>Secret</b>     | Same as ConfigMap but for sensitive data like passwords. Values are base64-encoded.                                   |
| <b>kubectl</b>    | The command-line tool to talk to your cluster. Like git but for Kubernetes.                                           |

## 1.3 Kubernetes Architecture

A Kubernetes cluster has two types of machines: the **Control Plane** (the brain) and **Worker Nodes** (the muscle). Here is what lives inside each:

| Control Plane Components    | What It Does                                                           |
|-----------------------------|------------------------------------------------------------------------|
| API Server (kube-apiserver) | The front door. Every kubectl command talks to this.                   |
| Scheduler (kube-scheduler)  | Decides which node a new pod should run on.                            |
| Controller Manager          | Watches the cluster and fixes drift (e.g. pod died → start a new one). |
| etcd                        | The database. Stores ALL cluster state. Very important — back it up!   |
| Worker Node Components      | What It Does                                                           |
| kubelet                     | The agent on each node. Gets orders from API server and runs pods.     |
| kube-proxy                  | Handles networking rules so services work correctly.                   |
| Container Runtime           | Actually runs the containers (e.g. containerd, Docker).                |

### ■ Note

When you run **kubectl apply -f mypod.yaml**, here is the journey: kubectl → API Server → Scheduler picks a node → kubelet on that node → Container Runtime runs the container. That's the whole loop!

## 1.4 Installing Your Local Cluster

You have two great options for a local cluster. Both are free and run on your laptop:

|              | minikube                  | kind (Kubernetes IN Docker) |
|--------------|---------------------------|-----------------------------|
| Best for     | Beginners, GUI dashboards | CI environments, multi-node |
| Runs on      | VM or Docker              | Docker containers only      |
| Install time | ~5 min                    | ~2 min                      |
| Recommended? | ■ Yes — start here        | Later for CI pipelines      |

## Installing minikube (Recommended for Beginners)

```
# macOS — install with Homebrew
brew install minikube
minikube start --driver=docker
minikube status
```

```
# Windows — install with Chocolatey (run as Administrator)
choco install minikube
minikube start --driver=docker
minikube status
```

```
# Linux (Ubuntu/Debian)
curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
sudo install minikube-linux-amd64 /usr/local/bin/minikube
minikube start --driver=docker
minikube status
```

### ■ Tip

If you see **minikube status** showing **Running** for all components, you are good to go! If Docker is not installed, install Docker Desktop first from [docker.com](https://www.docker.com).

## Installing kubectl (The CLI Tool)

```
# macOS
brew install kubectl
kubectl version --client
```

```
# Windows (PowerShell)
choco install kubernetes-cli
```

```
kubectl version --client
```

```
# Linux
sudo apt-get update && sudo apt-get install -y kubectl
kubectl version --client
```

## 1.5 Your First kubectl Commands

These are the 20 commands you will use 90% of the time. Memorise these and you are set.

| Command                                                         | What It Does                                               |
|-----------------------------------------------------------------|------------------------------------------------------------|
| <code>kubectl get nodes</code>                                  | List all nodes in the cluster                              |
| <code>kubectl get pods</code>                                   | List all pods in default namespace                         |
| <code>kubectl get pods -n test-ns</code>                        | List pods in 'test-ns' namespace                           |
| <code>kubectl get pods --all-namespaces</code>                  | List pods in every namespace                               |
| <code>kubectl get deployments</code>                            | List all deployments                                       |
| <code>kubectl get services</code>                               | List all services                                          |
| <code>kubectl describe pod &lt;pod-name&gt;</code>              | Full details about a pod (events, errors)                  |
| <code>kubectl logs &lt;pod-name&gt;</code>                      | Print logs from a pod                                      |
| <code>kubectl logs &lt;pod-name&gt; -f</code>                   | Stream logs live (like tail -f)                            |
| <code>kubectl logs &lt;pod-name&gt; -c &lt;container&gt;</code> | Logs from a specific container in a pod                    |
| <code>kubectl apply -f file.yaml</code>                         | Create or update resources from a YAML file                |
| <code>kubectl delete -f file.yaml</code>                        | Delete resources defined in a YAML file                    |
| <code>kubectl delete pod &lt;pod-name&gt;</code>                | Delete a specific pod (it will restart if in a Deployment) |
| <code>kubectl exec -it &lt;pod&gt; -- bash</code>               | SSH into a running pod                                     |
| <code>kubectl port-forward pod/&lt;pod&gt; 4444:4444</code>     | Forward port 4444 from pod to your localhost               |
| <code>kubectl scale deploy &lt;name&gt; --replicas=3</code>     | Scale a deployment to 3 replicas                           |
| <code>kubectl rollout status deploy/&lt;name&gt;</code>         | Watch a deployment rollout                                 |
| <code>kubectl config get-contexts</code>                        | List all cluster contexts (local, staging, prod...)        |
| <code>kubectl config use-context minikube</code>                | Switch to the minikube context                             |

---

```
kubectl get events --sort-by=.lastTimestamp
```

Recent cluster events (great for debugging)

### ■ Hands-On Lab: Hello Kubernetes!

**Step 1.** Start minikube: **minikube start --driver=docker**

**Step 2.** Verify cluster: **kubectl get nodes** — you should see 1 node with status Ready

**Step 3.** Run your first pod: **kubectl run hello-k8s --image=nginx --port=80**

**Step 4.** Watch it start: **kubectl get pods -w** (press Ctrl+C when status shows Running)

**Step 5.** See pod details: **kubectl describe pod hello-k8s**

**Step 6.** View logs: **kubectl logs hello-k8s**

**Step 7.** Forward a port: **kubectl port-forward pod/hello-k8s 8080:80**

**Step 8.** Open browser and visit <http://localhost:8080> — you should see the nginx welcome page!

**Step 9.** Clean up: **kubectl delete pod hello-k8s**

# Phase 2

## Core Kubernetes Objects Pods, Deployments, Services, ConfigMaps

### 2.1 Pods — The Atom of Kubernetes

A Pod is the smallest thing Kubernetes manages. It wraps one (or sometimes a few) containers and gives them a shared network and storage. You rarely create Pods directly — you use Deployments instead. But knowing how a raw Pod works helps you debug.

```
# pod.yaml — a simple nginx pod
apiVersion: v1
kind: Pod
metadata:
  name: my-test-pod
  labels:
    app: test-runner
    env: staging
spec:
  containers:
    - name: test-container
      image: nginx:latest
      ports:
        - containerPort: 80
      env:
        - name: ENVIRONMENT
          value: "staging"
      resources:
        requests:
          memory: "64Mi"
          cpu: "250m"
        limits:
          memory: "128Mi"
          cpu: "500m"
```

#### ■ Note

#### Line-by-line explanation:

- **apiVersion: v1** — tells K8s which API version to use for this object
- **kind: Pod** — the type of object we are creating
- **metadata.labels** — key-value tags for selecting/filtering pods
- **spec.containers** — list of containers inside this pod
- **resources.requests** — minimum resources this container needs
- **resources.limits** — maximum resources it can use (prevents hogging)

#### Warning

Pods are **ephemeral** — they die and their IP address changes. Never hardcode a pod IP. Always use Services to reach pods.

## 2.2 Deployments — Self-Healing Apps

A Deployment says: *'I want exactly N copies of this pod always running.'* If a pod crashes, the Deployment controller instantly starts a replacement. This is perfect for keeping your Selenium hub or test services always alive.

```
# deployment.yaml — selenium hub deployment
apiVersion: apps/v1
kind: Deployment
metadata:
  name: selenium-hub
  namespace: test-grid
spec:
  replicas: 1
  selector:
    matchLabels:
      app: selenium-hub
  template:
    metadata:
      labels:
        app: selenium-hub
    spec:
      containers:
        - name: selenium-hub
          image: selenium/hub:4.15
          ports:
            - containerPort: 4442
            - containerPort: 4443
            - containerPort: 4444
          env:
            - name: SE_SESSION_REQUEST_TIMEOUT
```

```

value: "300"
resources:
  requests:
    memory: "512Mi"
    cpu: "0.5"
  limits:
    memory: "1Gi"
    cpu: "1.0"

```

### ■ Key Concept

**selector.matchLabels** is the magic glue. It tells the Deployment: 'Manage all pods that have the label **app: selenium-hub**'. The **template.metadata.labels** must match this exactly — otherwise K8s won't know which pods belong to this Deployment.

```

# Common deployment commands
# Apply the deployment
kubectl apply -f deployment.yaml

# Check status
kubectl rollout status deployment/selenium-hub -n test-grid

# Scale up (e.g., run more test nodes)
kubectl scale deployment/selenium-hub --replicas=3 -n test-grid

# Update the image (triggers rolling update)
kubectl set image deployment/selenium-hub selenium-hub=selenium/hub:4.16 -n test-grid

# Roll back if something broke
kubectl rollout undo deployment/selenium-hub -n test-grid

```

## 2.3 Services — Networking Made Simple

As we said, pod IPs change every time a pod restarts. A Service gives you a **stable IP and DNS name** that always points to the right pods (using label selectors under the hood).

| Service Type | Use Case                | Accessible From         |
|--------------|-------------------------|-------------------------|
| ClusterIP    | Internal only (default) | Inside the cluster      |
| NodePort     | Dev/testing access      | Outside via nodeIP:port |
| LoadBalancer | Cloud production        | Outside via cloud LB IP |

```

# service.yaml — expose selenium hub
apiVersion: v1
kind: Service

```

```
metadata:
  name: selenium-hub-svc
  namespace: test-grid
spec:
  type: NodePort # Use ClusterIP for internal-only
  selector:
    app: selenium-hub # Matches pods with this label
  ports:
    - name: hub-port
  protocol: TCP
  port: 4444 # Port inside cluster
  targetPort: 4444 # Port inside the pod
  nodePort: 30444 # Port on your machine (30000-32767)
```

#### ■ Tip

On minikube, run **minikube service selenium-hub-svc -n test-grid --url** to get the URL to access your service from the browser or test code.

## 2.4 Namespaces — Keeping Things Organised

A Namespace is a virtual boundary inside your cluster. It's like having separate folders for dev, staging, and test environments on the same machine. Resources in different namespaces cannot accidentally interfere with each other.

```
# Create namespaces for your test environments
# Create namespaces
kubectl create namespace test-grid
kubectl create namespace staging
kubectl create namespace monitoring

# Or use a YAML file
---
apiVersion: v1
kind: Namespace
metadata:
  name: test-grid
  labels:
    purpose: automation
# List all namespaces
kubectl get namespaces

# Set a default namespace so you don't type -n every time
kubectl config set-context --current --namespace=test-grid
```

---

## 2.5 ConfigMaps & Secrets

Never hardcode environment-specific values (URLs, timeouts, credentials) in your Docker image. Use ConfigMaps for non-sensitive data and Secrets for sensitive data.

```
# configmap.yaml – test configuration
apiVersion: v1
kind: ConfigMap
metadata:
  name: test-config
  namespace: test-grid
data:
  BASE_URL: "https://staging.myapp.com"
  BROWSER: "chrome"
  TIMEOUT: "30"
  PARALLEL_THREADS: "5"
  SCREENSHOT_ON_FAIL: "true"
```

```
# secret.yaml – sensitive credentials
apiVersion: v1
kind: Secret
metadata:
  name: test-credentials
  namespace: test-grid
type: Opaque
data:
  # Values must be base64 encoded
  # echo -n 'admin' | base64 → YWRtaW4=
  DB_PASSWORD: YWRtaW4=
  API_KEY: c2VjcjcmV0a2V5MTIz
```

```
# Consume ConfigMap and Secret in a pod
spec:
  containers:
    - name: test-runner
      image: my-test-image:latest
      envFrom:
        - configMapRef:
            name: test-config # Loads ALL keys as env vars
        - secretRef:
            name: test-credentials # Loads ALL secrets as env vars
      # Or pick specific keys:
      env:
        - name: MY_URL
      valueFrom:
```

```
configMapKeyRef:  
  name: test-config  
  key: BASE_URL
```

### ■ Hands-On Lab: Deploy a Test Application

**Step 1.** Create a namespace: **kubectl create namespace test-grid**

**Step 2.** Save the ConfigMap YAML above and apply: **kubectl apply -f configmap.yaml**

**Step 3.** Create a Deployment YAML for selenium-hub (from section 2.2) and apply it

**Step 4.** Create a Service YAML (from section 2.3) and apply it

**Step 5.** Verify everything: **kubectl get all -n test-grid**

**Step 6.** Describe the deployment: **kubectl describe deployment selenium-hub -n test-grid**

**Step 7.** Get the URL: **minikube service selenium-hub-svc -n test-grid --url**

**Step 8.** Open the URL in browser — you should see the Selenium Grid UI

**Step 9.** Scale up nodes: **kubectl scale deployment/selenium-hub --replicas=2 -n test-grid**

**Step 10.** Watch pods: **kubectl get pods -n test-grid -w**

## Phase 3

## Running Test Automation Workloads Jobs, Selenium Grid, Playwright, Reports

### 3.1 Kubernetes Jobs & CronJobs for Tests

A **Deployment** is for long-running services (keep this pod alive forever). A **Job** is for one-time tasks (run this pod, complete the task, then exit). Test suites are **perfect** for Jobs — run, report results, done.

#### ■ Key Concept

##### Job vs Deployment:

- Deployment: 'Keep nginx running forever, restart if it dies'
- Job: 'Run my 200 Selenium tests once, wait for completion, do not restart'

```
# job.yaml — run a test suite as a Kubernetes Job
apiVersion: batch/v1
kind: Job
metadata:
  name: regression-suite
  namespace: test-grid
spec:
  completions: 1 # Total successful completions needed
  parallelism: 1 # How many pods run at the same time
  backoffLimit: 2 # Retry up to 2 times on failure
  activeDeadlineSeconds: 1800 # Kill job if it runs > 30 min
  template:
    metadata:
      labels:
        job: regression-suite
    spec:
      restartPolicy: Never # IMPORTANT: Jobs must use Never or OnFailure
      containers:
        - name: test-runner
          image: my-selenium-tests:latest
          envFrom:
            - configMapRef:
                name: test-config
          volumeMounts:
            - name: reports
              mountPath: /app/test-reports
      volumes:
```

```
- name: reports
emptyDir: {} # Temporary storage (use PVC for persistent)
```

#### ■ Tip

**restartPolicy: Never** is mandatory for Jobs. If you use **Always** (the default for Deployments), K8s will restart your test runner in a loop forever!

## Parallel Test Execution with Jobs

You can split your test suite and run them in parallel by setting **parallelism** and **completions** together:

```
# parallel-job.yaml – run 4 test shards simultaneously
spec:
  completions: 4 # Need 4 pods to complete successfully
  parallelism: 4 # Run all 4 at the same time
  template:
    spec:
      restartPolicy: Never
      containers:
        - name: test-shard
          image: my-tests:latest
          env:
            - name: JOB_COMPLETION_INDEX # K8s sets this automatically (0,1,2,3)
              valueFrom:
                fieldRef:
                  fieldPath: metadata.annotations['batch.kubernetes.io/job-completion-index']
            - name: TOTAL_SHARDS
              value: "4"
```

## CronJob — Scheduled Test Runs

```
# cronjob.yaml – run smoke tests every hour
apiVersion: batch/v1
kind: CronJob
metadata:
  name: smoke-tests-hourly
  namespace: test-grid
spec:
  schedule: "0 * * * *" # Every hour at minute 0 (cron syntax)
  concurrencyPolicy: Forbid # Don't start new run if previous still running
  successfulJobsHistoryLimit: 3
  failedJobsHistoryLimit: 3
  jobTemplate:
    spec:
```

```
template:
spec:
  restartPolicy: Never
  containers:
  - name: smoke-runner
    image: my-tests:latest
    args: ["--suite", "smoke"]
```

■ **Note**

### Cron Schedule Quick Reference:

- **0 \*\*\*** — every hour
- **0 0 \*\*\*** — every day at midnight
- **0 9 \*\* 1-5** — weekdays at 9am
- **\*/15 \*\*\*** — every 15 minutes

## 3.2 Full Selenium Grid on Kubernetes

Selenium Grid 4 has three components: **Hub** (the router), **Chrome nodes**, and **Firefox nodes**. Each becomes a Deployment in Kubernetes. Let's set up the whole grid.

```
# selenium-grid-complete.yaml - production-ready grid  
# ■■■ Namespace ████████████████████████████████████████████████████████████  
  
apiVersion: v1  
kind: Namespace  
metadata:  
name: selenium-grid  
---  
  
# ■■■ Hub Deployment ████████████████████████████████████████████████████████  
  
apiVersion: apps/v1  
kind: Deployment  
metadata:  
name: selenium-hub  
namespace: selenium-grid  
spec:  
replicas: 1  
selector:  
matchLabels:  
app: selenium-hub  
template:  
metadata:  
labels:  
app: selenium-hub  
spec:  
containers:
```

```
- name: selenium-hub  
image: selenium/hub:4.15  
ports:  
- containerPort: 4444  
  
resources:  
requests:  
memory: "512Mi"  
cpu: "500m"  
limits:  
memory: "1Gi"  
cpu: "1000m"  
---  
  
# ■■ Hub Service ████████████████████████████████████████████████████████████  
apiVersion: v1  
kind: Service  
metadata:  
name: selenium-hub  
namespace: selenium-grid  
spec:  
type: NodePort  
selector:  
app: selenium-hub  
ports:  
- port: 4444  
targetPort: 4444  
nodePort: 30444  
---  
  
# ■■ Chrome Node Deployment ████████████████████████████████████████████████████████████  
apiVersion: apps/v1  
kind: Deployment  
metadata:  
name: selenium-chrome  
namespace: selenium-grid  
spec:  
replicas: 3 # 3 Chrome instances in parallel  
selector:  
matchLabels:  
app: selenium-chrome  
template:  
metadata:  
labels:  
app: selenium-chrome  
spec:  
containers:  
- name: selenium-chrome
```

```
image: selenium/node-chrome:4.15
env:
  - name: SE_EVENT_BUS_HOST
    value: "selenium-hub"
  - name: SE_EVENT_BUS_PUBLISH_PORT
    value: "4442"
  - name: SE_EVENT_BUS_SUBSCRIBE_PORT
    value: "4443"
  - name: SE_NODE_MAX_SESSIONS
    value: "1"
resources:
  requests:
    memory: "1Gi"
    cpu: "0.5"
  limits:
    memory: "2Gi"
    cpu: "1.5"
  volumeMounts:
    - mountPath: /dev/shm
  name: dshm
  volumes:
    - name: dshm # Shared memory – Chrome needs this!
emptyDir:
medium: Memory
sizeLimit: 2Gi
```

```
# Deploy and verify the grid
# Apply everything
kubectl apply -f selenium-grid-complete.yaml

# Watch pods start up
kubectl get pods -n selenium-grid -w

# Get grid URL on minikube
minikube service selenium-hub -n selenium-grid --url

# Verify grid status (open in browser → you should see 3 Chrome nodes registered)
# http://localhost:30444/ui

# Point your Java/Python tests to this URL:
# WebDriver driver = new RemoteWebDriver(
#   new URL("http://localhost:30444"),
#   new ChromeOptions()
# )
```

### 3.3 Playwright in Kubernetes

Playwright is simpler to containerise than Selenium because it bundles its own browsers. You just need a Docker image with Playwright installed.

```
# Dockerfile for Playwright tests
FROM mcr.microsoft.com/playwright:v1.40.0-jammy

WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .

# Default: run all tests
CMD ["npm", "run", "test", "--reporter=html"]
```

```
# playwright-job.yaml – run Playwright tests as a K8s Job
apiVersion: batch/v1
kind: Job
metadata:
  name: playwright-tests
  namespace: test-grid
spec:
  completions: 1
  backoffLimit: 1
  template:
    spec:
      restartPolicy: Never
      containers:
        - name: playwright
          image: my-playwright-tests:latest
          env:
            - name: BASE_URL
              valueFrom:
                configMapKeyRef:
                  name: test-config
                  key: BASE_URL
            - name: CI
              value: "true"
          resources:
            requests:
              memory: "1Gi"
              cpu: "0.5"
            limits:
              memory: "2Gi"
              cpu: "2"
          volumeMounts:
            - name: test-results
              mountPath: /app/playwright-report
```

```
volumes:
- name: test-results
persistentVolumeClaim:
claimName: test-reports-pvc
```

### 3.4 Persistent Volumes for Test Reports

When a Job pod finishes, its storage is deleted. To save test reports and screenshots, use a **PersistentVolumeClaim (PVC)** which gives you storage that survives pod restarts.

```
# pvc.yaml - claim persistent storage for test reports
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
name: test-reports-pvc
namespace: test-grid
spec:
accessModes:
- ReadWriteOnce # One pod can write at a time
resources:
requests:
storage: 5Gi # Request 5GB of storage
storageClassName: standard # Use 'standard' on minikube
```

```
# Access reports after tests complete
# Copy reports from the PVC using a helper pod
kubectl run report-viewer --image=nginx --restart=Never -n test-grid --overrides='{
  "spec": {
    "volumes": [
      {
        "name": "reports",
        "persistentVolumeClaim": {
          "claimName": "test-reports-pvc"
        }
      }
    ],
    "containers": [
      {
        "name": "nginx",
        "image": "nginx",
        "volumeMounts": [
          {
            "mountPath": "/usr/share/nginx/html",
            "name": "reports"
          }
        ]
      }
    ]
  }
}'

# Forward the port to view reports in browser
kubectl port-forward pod/report-viewer 8090:80 -n test-grid

# Open http://localhost:8090 to see your HTML test reports
```

#### ■ Hands-On Lab: Run Selenium Tests in a Kubernetes Job

**Step 1.** Deploy the full Selenium Grid: **kubectl apply -f selenium-grid-complete.yaml**

**Step 2.** Wait for all pods Ready: **kubectl get pods -n selenium-grid**

**Step 3.** Get the grid URL: **minikube service selenium-hub -n selenium-grid --url**

**Step 4.** Create a simple Java/Python test that targets RemoteWebDriver at the grid URL

**Step 5.** Build a Docker image from your test: **docker build -t my-tests:v1 .**

---

**Step 6.** Load image into minikube: **minikube image load my-tests:v1**

**Step 7.** Apply the Job YAML: **kubectl apply -f job.yaml**

**Step 8.** Watch the job: **kubectl get jobs -n test-grid -w**

**Step 9.** View test logs: **kubectl logs -l job=regression-suite -n test-grid**

**Step 10.** Check final status: **kubectl describe job regression-suite -n test-grid**

## Phase 4

## CI/CD Integration GitHub Actions, Helm, Test Reporting

### 4.1 GitHub Actions + kubectl

The most common CI/CD pattern for Kubernetes is: push code → GitHub Actions builds Docker image → pushes to registry → runs kubectl/helm to deploy updated tests.

```
# .github/workflows/test-pipeline.yml
name: Test Automation Pipeline

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]
  schedule:
    - cron: '0 2 * * *' # Also run nightly at 2 AM

env:
  REGISTRY: ghcr.io
  IMAGE_NAME: ${ github.repository }/test-runner

jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4
      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v3
      - name: Login to GitHub Container Registry
        uses: docker/login-action@v3
        with:
          registry: ${ env.REGISTRY }
          username: ${ github.actor }
          password: ${ secrets.GITHUB_TOKEN }
      - name: Build and push Docker image
        uses: docker/build-push-action@v5
        with:
          context: .
          push: true
          tags: |
```

```

${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}:${{ github.sha }}
${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}:latest
- name: Set up kubectl
uses: azure/setup-kubectl@v3
with:
version: 'v1.28.0'
- name: Configure Kubeconfig
run: |
mkdir -p $HOME/.kube
echo "${{ secrets.KUBECONFIG }}" | base64 -d > $HOME/.kube/config
- name: Update image tag in Job YAML
run: |
sed -i "s|my-tests:latest|${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}:${{ github.sha }}|g" k8s/job.yaml
- name: Deploy test Job
run: |
kubectl apply -f k8s/job.yaml
kubectl wait --for=condition=complete job/regression-suite -n test-grid --timeout=1800s
- name: Fetch test results
if: always()
run: |
POD=$(kubectl get pods -n test-grid -l job=regression-suite -o
jsonpath='{.items[0].metadata.name}')
kubectl cp test-grid/$POD:/app/test-reports ./test-reports
- name: Upload test reports
if: always()
uses: actions/upload-artifact@v4
with:
name: test-reports-${{ github.sha }}
path: ./test-reports/
retention-days: 30
- name: Cleanup Job
if: always()
run: kubectl delete job regression-suite -n test-grid --ignore-not-found=true

```

#### ■ Tip

Store your kubeconfig as a base64 GitHub Secret: **cat ~/.kube/config | base64 | pbcopy** then paste it as secret **KUBECONFIG**

## 4.2 Helm — The Package Manager for Kubernetes

Helm is to Kubernetes what npm is to Node.js. Instead of managing 10 separate YAML files, you bundle them into a **Helm Chart** and deploy everything with one command. It also handles versioning and

rollbacks cleanly.

```
# Install Helm and add common repositories
# macOS
brew install helm

# Linux
curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash

# Add popular chart repositories
helm repo add stable https://charts.helm.sh/stable
helm repo add selenium https://www.selenium.dev/docker-selenium-grid/charts
helm repo update

# Search for charts
helm search repo selenium
```

```
# Deploy Selenium Grid with Helm (much easier!)
# Install Selenium Grid with one command
helm install selenium-grid selenium/selenium-grid --namespace selenium-grid
--create-namespace --set hub.serviceType=NodePort --set chromeNode.replicas=3 --set
firefoxNode.replicas=2

# Check status
helm status selenium-grid -n selenium-grid

# Upgrade (change chrome replicas to 5)
helm upgrade selenium-grid selenium/selenium-grid --namespace selenium-grid --set
chromeNode.replicas=5

# Roll back to previous version
helm rollback selenium-grid 1 -n selenium-grid

# Uninstall everything
helm uninstall selenium-grid -n selenium-grid
```

## Creating Your Own Helm Chart

```
# Create a chart scaffold for your test automation
# Create chart structure
helm create test-automation

# Structure created:
# test-automation/
# Chart.yaml ← Chart metadata
# values.yaml ← Default config values
# templates/
# deployment.yaml ← Template files with {{ }} placeholders
# service.yaml
# job.yaml
# configmap.yaml
```

```
# values.yaml – centralised configuration
# values.yaml
image:
repository: my-tests
tag: latest
pullPolicy: IfNotPresent
testConfig:
baseUrl: "https://staging.myapp.com"
browser: "chrome"
timeout: "30"
parallelThreads: "3"
job:
completions: 1
parallelism: 1
backoffLimit: 2
resources:
requests:
memory: "1Gi"
cpu: "500m"
limits:
memory: "2Gi"
cpu: "1000m"
```

```
# Install your chart with environment overrides
# Install for staging
helm install regression-staging ./test-automation --set
testConfig.baseUrl=https://staging.myapp.com --set testConfig.parallelThreads=3
# Install for production with separate values file
helm install regression-prod ./test-automation -f values-production.yaml
```

## 4.3 Test Result Artefacts & Reporting

Collecting test results from Kubernetes pods is a common pain point. Here are the three main patterns:

| Pattern          | How                               | Best For                     |
|------------------|-----------------------------------|------------------------------|
| kubectl cp       | Copy files from pod after test    | Simple setups, small reports |
| PVC + report pod | Shared volume, spin up nginx pod  | Long-running clusters        |
| S3/GCS upload    | Test code uploads to cloud bucket | Production pipelines         |

```
# Pattern 1: kubectl cp (simplest)
```

```
# Wait for job to finish
kubectl wait --for=condition=complete job/my-tests -n test-grid --timeout=600s

# Find the pod name
POD=$(kubectl get pods -n test-grid -l job-name=my-tests -o
jsonpath='{.items[0].metadata.name}')

# Copy report files to local
kubectl cp test-grid/$POD:/app/reports ./local-reports

# Copy a specific file
kubectl cp test-grid/$POD:/app/report.html ./report.html
```

### ■ Hands-On Lab: Build a Full CI/CD Pipeline

- Step 1.** Create a GitHub repository for your test project
- Step 2.** Write a Dockerfile for your Playwright/Selenium tests
- Step 3.** Create the k8s/ directory with your job.yaml and configmap.yaml
- Step 4.** Copy the GitHub Actions workflow from section 4.1 into .github/workflows/test-pipeline.yml
- Step 5.** Set up GitHub Secrets: KUBECONFIG (base64 encoded kubeconfig)
- Step 6.** Push to GitHub — the workflow should trigger automatically
- Step 7.** Watch the Actions run in GitHub → Actions tab
- Step 8.** Download the test-reports artifact from the completed workflow
- Step 9.** Try a failing test to see how the pipeline handles it
- Step 10.** Add Helm: **helm create test-automation**, fill in the templates, and deploy via helm

# Phase 5

## Production Readiness Resources, Health Checks, Monitoring & Debugging

### 5.1 Resource Limits & Requests

If you don't set resource limits, one misbehaving test pod can consume all cluster memory and crash everything. Always define **requests** (minimum guaranteed) and **limits** (hard ceiling).

```
# Resource configuration guide
spec:
  containers:
  - name: test-runner
    resources:
      requests:
        memory: "256Mi" # K8s guarantees this much memory
        cpu: "250m" # 250 millicores = 0.25 CPU core
      limits:
        memory: "512Mi" # Pod is KILLED if it exceeds this
        cpu: "500m" # Pod is THROTTLED if it exceeds this
# Common values for test automation:
# Light test runner: requests 256Mi/0.25cpu limits 512Mi/0.5cpu
# Selenium node: requests 1Gi/0.5cpu limits 2Gi/1.5cpu
# Playwright runner: requests 1Gi/0.5cpu limits 2Gi/2cpu
```

#### Warning

CPU is **compressible** (pod slows down when throttled). Memory is **incompressible** (pod is killed with OOMKilled error if it exceeds limits). Always set memory limits carefully!

### ResourceQuota — Limit total resources per namespace

```
# resource-quota.yaml
apiVersion: v1
kind: ResourceQuota
metadata:
  name: test-grid-quota
  namespace: test-grid
spec:
  hard:
    pods: "20" # Max 20 pods in this namespace
```

```
requests.cpu: "8" # Total CPU requested across all pods
requests.memory: 16Gi
limits.cpu: "16"
limits.memory: 32Gi
```

## 5.2 Health Checks — Liveness & Readiness Probes

Kubernetes can automatically detect when your pods are unhealthy and restart them. Two types of probes handle this:

| Probe Type     | Question It Answers                      | Action on Failure                          |
|----------------|------------------------------------------|--------------------------------------------|
| livenessProbe  | Is the container alive (not hung)?       | Restart the container                      |
| readinessProbe | Is the container ready to serve traffic? | Remove from Service (stop sending traffic) |
| startupProbe   | Has the app finished starting up?        | Keep waiting (don't restart yet)           |

```
# Probes on Selenium Hub
spec:
  containers:
  - name: selenium-hub
    image: selenium/hub:4.15
    livenessProbe:
      httpGet:
        path: /wd/hub/status # Selenium's health endpoint
        port: 4444
      initialDelaySeconds: 30 # Wait 30s before first check
      periodSeconds: 10 # Check every 10s
      failureThreshold: 3 # Restart after 3 consecutive failures
    readinessProbe:
      httpGet:
        path: /wd/hub/status
        port: 4444
      initialDelaySeconds: 15
      periodSeconds: 5
      failureThreshold: 2 # Remove from Service after 2 failures
    startupProbe:
      httpGet:
        path: /wd/hub/status
        port: 4444
      failureThreshold: 12 # Allow up to 12*5=60s for startup
      periodSeconds: 5
```

## 5.3 Monitoring with Prometheus & Grafana

For production test clusters, you want to know: How many tests are running? How long do they take? How much memory are nodes using? Prometheus collects metrics and Grafana visualises them.

```
# Install Prometheus + Grafana stack with Helm
# Add the Prometheus community chart
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo update

# Install the full stack (Prometheus + Grafana + Alertmanager)
helm install monitoring prometheus-community/kube-prometheus-stack --namespace
monitoring --create-namespace --set grafana.adminPassword=testpass123 --set
prometheus.prometheusSpec.serviceMonitorSelectorNilUsesHelmValues=false

# Access Grafana dashboard
kubectl port-forward svc/monitoring-grafana 3000:80 -n monitoring

# Open http://localhost:3000 (admin / testpass123)

# Access Prometheus
kubectl port-forward svc/monitoring-kube-prometheus-prometheus 9090:9090 -n monitoring
```

### ■ Tip

In Grafana, search for '**Kubernetes / Compute Resources / Namespace**' dashboard. It shows CPU and memory for your test-grid namespace in real time.

## 5.4 Debugging Cheat Sheet

When things go wrong (and they will — that's normal!), here is your debugging playbook:

### ■ Symptom: Pod stuck in Pending

**Command:** `kubectl describe pod | grep -A5 Events`

**Likely cause:** Usually: not enough CPU/memory on nodes, or PVC not bound

### ■ Symptom: Pod in CrashLoopBackOff

**Command:** `kubectl logs --previous`

**Likely cause:** The container crashed. `--previous` shows logs from the crashed instance

### ■ Symptom: Pod in ImagePullBackOff

**Command:** `kubectl describe pod`

**Likely cause:** Image name wrong, or registry credentials missing (create an `imagePullSecret`)

### ■ Symptom: OOMKilled (Out of Memory)

**Command:** `kubectl describe pod | grep -i oom`

**Likely cause:** Increase memory limits or reduce test parallelism

#### ■ Symptom: Tests can't reach Selenium Hub

**Command:** `kubectl exec -it -- curl http://selenium-hub:4444/wd/hub/status`

**Likely cause:** Check Service name, namespace, and label selectors match

#### ■ Symptom: Job never completes

**Command:** `kubectl describe job && kubectl get events -n test-grid`

**Likely cause:** Check activeDeadlineSeconds and pod logs for hanging tests

```
# Quick debug one-liners
# Get all failing pods
kubectl get pods --all-namespaces | grep -v Running | grep -v Completed
# See recent cluster-wide events (sorted newest first)
kubectl get events --all-namespaces --sort-by='.lastTimestamp' | tail -30
# Check resource usage on nodes
kubectl top nodes
# Check resource usage on pods
kubectl top pods -n test-grid
# Force delete a stuck pod
kubectl delete pod <name> -n test-grid --force --grace-period=0
# Debug networking: run a busybox pod and curl from inside
kubectl run debug --image=busybox --rm -it -- sh
```

### ■ Hands-On Lab: Production-Grade Test Cluster

**Step 1.** Add resource requests+limits to all your Deployments and Jobs

**Step 2.** Add a livenessProbe and readinessProbe to your Selenium Hub deployment

**Step 3.** Apply a ResourceQuota to the test-grid namespace

**Step 4.** Install Prometheus+Grafana: **helm install monitoring prometheus-community/kube-prometheus-stack -n monitoring --create-namespace**

**Step 5.** Port-forward Grafana and explore the Kubernetes dashboards

**Step 6.** Intentionally break something (wrong image name) and practice debugging with describe/logs

**Step 7.** Set memory limit too low (e.g. 50Mi for Chrome) and observe the OOMKilled error

**Step 8.** Fix it and watch K8s recover automatically

**Step 9.** Run `kubectl top pods -n test-grid` while tests run — observe resource usage

---

**Step 10.** Celebrate! You now have a production-grade test automation cluster ■

---

# Quick-Reference Cards

---

## Every Kubernetes YAML Has This Structure

```
# universal K8s YAML skeleton
apiVersion: <api-group/version> # e.g. v1, apps/v1, batch/v1
kind: <ObjectType> # Pod, Deployment, Service, Job, etc.
metadata:
  name: <unique-name>
  namespace: <namespace> # omit for default namespace
  labels: # key-value tags – very important for selectors!
    key: value
  annotations: # metadata for tools (not for K8s itself)
    key: value
spec: # The desired state – unique per object type
  ...your config here...
```

## Label Selectors — The Glue of Kubernetes

```
# How Deployment finds its Pods
# The Deployment's selector
spec:
  selector:
    matchLabels:
      app: my-app # Must match...
# ...the Pod template's labels
template:
  metadata:
    labels:
      app: my-app # ...exactly this!
# The Service's selector
spec:
  selector:
    app: my-app # Routes to any pod with this label
```

## kubectl Master Cheat Sheet

| Category     | Command                                | Description              |
|--------------|----------------------------------------|--------------------------|
| Cluster Info | <code>kubectl cluster-info</code>      | Show cluster endpoints   |
|              | <code>kubectl get nodes -o wide</code> | Nodes with IPs and roles |

|                    |                                                                          |                           |
|--------------------|--------------------------------------------------------------------------|---------------------------|
|                    | <code>kubectl top nodes</code>                                           | Node CPU/memory usage     |
| <b>Pods</b>        | <code>kubectl get pods -n &lt;ns&gt; -o wide</code>                      | Pods with node placement  |
|                    | <code>kubectl describe pod &lt;name&gt; -n &lt;ns&gt;</code>             | Full pod details + events |
|                    | <code>kubectl logs &lt;pod&gt; -c &lt;container&gt; --tail=100</code>    | Last 100 log lines        |
|                    | <code>kubectl exec -it &lt;pod&gt; -- bash</code>                        | Shell into pod            |
|                    | <code>kubectl delete pod &lt;pod&gt; --force</code>                      | Force delete stuck pod    |
| <b>Deployments</b> | <code>kubectl get deploy -n &lt;ns&gt;</code>                            | List deployments          |
|                    | <code>kubectl rollout restart deploy/&lt;name&gt;</code>                 | Rolling restart           |
|                    | <code>kubectl rollout history deploy/&lt;name&gt;</code>                 | Rollout history           |
|                    | <code>kubectl rollout undo deploy/&lt;name&gt;</code>                    | Roll back one version     |
| <b>Jobs</b>        | <code>kubectl get jobs -n &lt;ns&gt;</code>                              | List all jobs             |
|                    | <code>kubectl wait --for=condition=complete job/&lt;name&gt;</code>      | Wait for job to finish    |
|                    | <code>kubectl delete job &lt;name&gt; -n &lt;ns&gt;</code>               | Delete a job              |
| <b>Services</b>    | <code>kubectl get svc -n &lt;ns&gt;</code>                               | List services             |
|                    | <code>kubectl port-forward svc/&lt;svc&gt; 8080:80</code>                | Port-forward a service    |
| <b>Config</b>      | <code>kubectl get configmap &lt;name&gt; -o yaml</code>                  | View configmap content    |
|                    | <code>kubectl get secret &lt;name&gt; -o yaml</code>                     | View secret (base64)      |
| <b>Namespaces</b>  | <code>kubectl get ns</code>                                              | List all namespaces       |
|                    | <code>kubectl config set-context --current --namespace=&lt;ns&gt;</code> | Set default namespace     |
| <b>Debugging</b>   | <code>kubectl get events --sort-by=.lastTimestamp</code>                 | Recent events             |
|                    | <code>kubectl top pods -n &lt;ns&gt;</code>                              | Pod resource usage        |
|                    | <code>kubectl describe node &lt;name&gt;</code>                          | Node details + conditions |

# Complete Glossary

|                   |                                                                               |
|-------------------|-------------------------------------------------------------------------------|
| <b>API Server</b> | The front door to the Kubernetes control plane. All kubectl commands go here. |
| <b>ClusterIP</b>  | Default Service type — only accessible inside the cluster.                    |
| <b>ConfigMap</b>  | Stores non-sensitive configuration as key-value pairs.                        |
| <b>Container</b>  | A runnable instance of a Docker image.                                        |

|                         |                                                                                     |
|-------------------------|-------------------------------------------------------------------------------------|
| <b>CronJob</b>          | Runs a Job on a scheduled (cron) basis.                                             |
| <b>Deployment</b>       | Manages a set of identical pods and ensures the desired count always runs.          |
| <b>DaemonSet</b>        | Ensures one pod runs on every node (e.g. log collectors).                           |
| <b>emptyDir</b>         | Temporary storage tied to a pod's lifetime — deleted when pod dies.                 |
| <b>etcd</b>             | The key-value database that stores all cluster state.                               |
| <b>Helm</b>             | The Kubernetes package manager. Bundles K8s resources into reusable charts.         |
| <b>Ingress</b>          | Manages HTTP/HTTPS external access to services (like an API gateway).               |
| <b>Job</b>              | Runs a pod once to completion (perfect for test suites).                            |
| <b>kind (tool)</b>      | 'Kubernetes IN Docker' — runs a cluster inside Docker containers. Great for CI.     |
| <b>kubectcl</b>         | The command-line tool to interact with a Kubernetes cluster.                        |
| <b>kubelet</b>          | The agent on each worker node that runs pods as instructed by the API server.       |
| <b>Labels</b>           | Key-value metadata attached to objects for selection and organisation.              |
| <b>Limits</b>           | Maximum resources a container can use (pod is killed/throttled if exceeded).        |
| <b>LivenessProbe</b>    | Checks if a container is still alive. Restarts it if the check fails.               |
| <b>LoadBalancer</b>     | Service type that provisions a cloud load balancer with an external IP.             |
| <b>minikube</b>         | Runs a single-node K8s cluster locally for development and learning.                |
| <b>Namespace</b>        | A virtual partition of the cluster for organising resources.                        |
| <b>Node</b>             | A machine (physical or VM) in the cluster that runs pods.                           |
| <b>NodePort</b>         | Service type exposing a port on every node for external access.                     |
| <b>OOMKilled</b>        | Out Of Memory Killed — a pod was killed because it exceeded memory limits.          |
| <b>PersistentVolume</b> | A piece of storage in the cluster with a lifecycle independent of pods.             |
| <b>PVC</b>              | PersistentVolumeClaim — a request for storage by a pod.                             |
| <b>Pod</b>              | The smallest deployable unit in K8s, holding one or more containers.                |
| <b>ReadinessProbe</b>   | Checks if a container is ready to serve traffic. Removes it from Service if not.    |
| <b>Replica</b>          | A copy of a pod. Deployments maintain the desired replica count.                    |
| <b>ReplicaSet</b>       | Ensures a specified number of pod replicas run at any time (managed by Deployment). |
| <b>Requests</b>         | Minimum resources guaranteed to a container by the scheduler.                       |
| <b>ResourceQuota</b>    | Limits total resources consumable by all pods in a namespace.                       |

---

|                    |                                                                                  |
|--------------------|----------------------------------------------------------------------------------|
| <b>Rollout</b>     | The process of gradually updating pods to a new version.                         |
| <b>Secret</b>      | Stores sensitive data (passwords, tokens) as base64-encoded key-value pairs.     |
| <b>Selector</b>    | A query using labels to identify a set of objects.                               |
| <b>Service</b>     | An abstract way to expose a set of pods as a stable network endpoint.            |
| <b>StatefulSet</b> | Like Deployment but for stateful apps (databases) — pods have stable identities. |
| <b>Volume</b>      | A directory accessible to containers in a pod.                                   |
| <b>YAML</b>        | The configuration language used for all Kubernetes resource definitions.         |